

# **cqs — Code Quality Studio CLI**

18 stacks · 533 rules · offline · CI-ready · Node 18+

INSTALL FROM NPM (ANYWHERE)

```
npm install -g cqs-audit
```

[npmjs.com/package/cqs-audit](https://npmjs.com/package/cqs-audit)

✓ Verify: **cqs --version**

**v2.6.0**

## 1 · Install & Everyday Use

---

Install → verify → run your first audit

## 2 AI-Assisted Review

### INSTALL FROM NPM (PUBLIC — ANY MACHINE)

#### AI SETUP — ANTHROPIC & OPENAI

**npm install -g cqs-audit**  
**ANTHROPIC (CLAUDE)**

```
# 1. Key from console.anthropic.com
# 2. Persist it in your shell profile
echo 'export ANTHROPIC_API_KEY=sk-ant-xxx' >>
~/.zshrc
source ~/.zshrc
# 3. Run — key picked up automatically
cqs ./tests/ --ai anthropic
```

**OPENAI (GPT)**

```
# 1. Key from platform.openai.com
echo 'export OPENAI_API_KEY=sk-xxx' >> ~/.zshrc
source ~/.zshrc
# 2. Run (default model gpt-4o-mini)
cqs ./tests/ --ai openai
```

Set the key **once** — every later run reads it from the environment.

### NEW IN 2.4.0 — ADOPT ON AN EXISTING SUITE

#### AI SETUP — BEDROCK & VERTEX

**AWS BEDROCK** **baseline** fails only on new findings — so it can date CI from day one

```
echo 'export AWS_REGION=us-east-1' >> ~/.zshrc
echo 'export AWS_ACCESS_KEY_ID=AKIAxxx' >>
~/.zshrc
echo 'export AWS_SECRET_ACCESS_KEY=xxx' >>
~/.zshrc
source ~/.zshrc
cqs ./tests/ --ai bedrock
```

**GOOGLE VERTEX AI** **json / summary** and when piped, so CI stays clean. **--no-report** prints out

```
# Service-account JSON from the GCP console
echo 'export VERTEX_PROJECT=my-proj' >> ~/.zshrc
echo 'export VERTEX_LOCATION=us-central1' >>
~/.zshrc
echo 'export
GOOGLE_APPLICATION_CREDENTIALS=~/.sa.json' >>
~/.zshrc
source ~/.zshrc
cqs ./tests/ --ai vertex
```

**Unaudited** — no error. Anything with no matching stack is listed at

```
cqs ./tests/ --ai anthropic --api-key sk-ant-xxx
cqs ./tests/ --ai openai --api-key sk-xxx
cqs ./tests/ --ai bedrock --aws-region us-east-1 \
--aws-access-key KEY --aws-secret-key SECRET
cqs ./tests/ --ai vertex --vertex-project proj \
--vertex-location us-central1 --vertex-key-file
sa.json
```

### Set up CI key once. Every run can include an AI second eye

### CORE COMMANDS

#### CORE AI COMMANDS (AFTER KEY IS SET)

**cqs .** Analyse current folder —

```
# Audit with AI second-eye review
cqs ./tests/ --stack playwright --ai anthropic
cqs ./tests/ --stack playwright --ai openai
cqs ./tests/ --stack playwright --ai bedrock
cqs ./tests/ --stack playwright --ai vertex
```

**cqs ./tests/ --severity critical** Show only critical findings

```
# Review only the critical files
cqs ./tests/ --ai anthropic --severity critical
```

**j** # AI review + open HTML report

```
cqs ./tests/ --ai anthropic --open
```

**s** # AI review + save JSON

```
cqs ./tests/ --ai anthropic --output json >
r.json
```

**PICK A STACK / MODEL** Terminal output only —

```
cqs ./tests/ --ai anthropic --model claude-
sonnet-5
cqs ./tests/ --ai openai --model gpt-4o
```

**cqs --read-report report.json** A review run after the offline rules and only on files that already have critical or warning findings. Prints summary of a saved JSON report

**cqs --read-report report.json --open** Open saved report as HTML in browser

**cqs --list-stacks** Show all 18 available stacks + IDs

**cqs ./tests/ --baseline-write .cqs-baseline.json** Record current findings as accepted

**cqs ./tests/ --baseline .cqs-baseline.json** Fail only on findings beyond the baseline

3 · CI/CD, Output Formats & Packaging

📋 ALL 18 STACKS (--STACK <ID>)

UI AUTOMATION

playwright

playwright\_java

playwright\_python

selenium\_java

selenium\_csharp

cypress

MOBILE / MODEL-BASED

appium\_java

toscala\_xml

API TESTING

restassured

karate

pytest\_api

postman

APPLICATION CODE

java\_api

java\_frontend

typescript

ts\_frontend

python\_api

python\_frontend

| CMD                                        | DESCRIPTION                                               |
|--------------------------------------------|-----------------------------------------------------------|
| <code>cqs --changed</code>                 | Audit only files changed vs <code>origin/main</code>      |
| <code>cqs --changed --since develop</code> | Choose the base ref to compare against                    |
| <code>cqs --version</code>                 | Print the installed version (also <code>-v</code> )       |
| <code>cqs mcp-config cursor</code>         | Print ready-to-paste MCP config (or <code>claude</code> ) |
| <code>cqs --help</code>                    | Show help text and version number                         |

## 4 IDE & AI Agent Integration

### CI / CD INTEGRATION

#### USING CQS WITH CURSOR IDE

# once — accept today's debt, commit the file  
**OPTION 1 — Terminal inside Cursor**

# Open terminal in Cursor (Ctrl+`), then run normally  
**cqs ./tests/ --stack playwright --open**  
# HTML report opens in your default browser

makes it worse

**OPTION 2 — Cursor Task (tasks.json)**  
**cqs --changed --baseline .cqs-baseline.json**

```
// .vscode/tasks.json
"tasks": [{
  "label": "cqs audit",
  "type": "shell",
  "command": "cqs ${workspaceFolder}/tests/ --open",
  "group": "test"
}]
```

#### OPTION 3 — Cursor AI + cqs together

1. Run: **cqs ./tests/ --output json > r.json**
2. Open **r.json** in Cursor
3. Ask Cursor AI: "Fix the critical findings in this report"
4. **Cursor** reads the JSON + your test files and applies fixes

#### PRE-COMMIT HOOK

```
cqs . --severity critical --output summary  
|| exit 1
```

## ADVANCED CHECKS — AUDITING AGAINST A RULE BOOK

### MCP SERVER — CQS AS AN AI TOOL

#### BUILD THE MCP SERVER

```
# From the cqs CLI source directory
cd code-quality-studio/cli
node build-mcp.mjs
# → produces dist/cqs-mcp.js
```

#### CONFIG — SAME JSON FOR BOTH CLIENTS

```
{
  "mcpServers": {
    "cqs": {
      "command": "node",
      "args": ["/abs/path/dist/cqs-mcp.js"]
    }
  }
}
```

Cursor → **.cursor/mcp.json**

Claude Code → **~/.claude/settings.json**

#### MCP TOOLS EXPOSED

**cqs\_audit** — audit a path, returns findings  
**cqs\_list\_stacks** — list all 18 supported stacks  
**cqs\_read\_report** — summarise a JSON report file

Once registered, ask the AI: "Audit my tests folder" — the AI calls cqs\_audit automatically.

this rule catch anything in ./tests:

## OUTPUT FORMATS & ADVANCED

**pretty** Colour terminal — scores, findings, bars (default)  
**json** Machine-readable — pipe to file for CI artefacts  
**summary** One-liner: score 90 · 5 critical · 12 warning

#### ADVANCED COMMANDS

Run cqs from Cursor, or expose it as a tool your AI agent can call

### CLAUDE CODE /CQS SKILL

#### INSTALL THE SKILL (one-time)

```
# Create the skill directory
mkdir -p ~/.claude/skills/cqs

# Copy SKILL.md from the cqs repo
cp cli/bin/cqs-skill.md
~/.claude/skills/cqs/SKILL.md
```

```
# Or download directly (if cqs-audit is installed globally)
cqs --version # verify cqs is working
```

#### USE IN CLAUDE CODE SESSION

```
# In any Claude Code session, just type:
/cqs

# Claude will ask you for the path, then run the audit
# and summarise findings directly in the conversation
```

#### EXAMPLE PROMPTS WITH SKILL

```
"Audit my tests and fix all critical issues"
"Run cqs on ./e2e/ and summarise what's wrong"
"Check my Playwright tests for quality issues"
"Review ./tests/ with AI — use Anthropic"
```

#### SKILL FILE LOCATION

```
~/.claude/skills/cqs/SKILL.md
```

Claude Code auto-loads skills from **~/.claude/skills/** — no restart needed after adding SKILL.md.

 REMEDIATION AGENT

Fixes findings in place. Every fix is re-audited before it is kept.

PREVIEW FIRST

```
cqs remediate ./tests/ --ai anthropic \
  --dry-run
# prints the diff, writes nothing
```

APPLY

```
cqs remediate ./tests/ --ai anthropic

# apply on a branch and commit
cqs remediate ./tests/ --ai anthropic \
  --branch fix/cqs --commit
```

FLAGS

- `--severity` what to fix (default critical)
- `--max-files` cap per run (default 10)
- `--force` write even if git is dirty
- `--provider` alias for `--ai`

 PR REVIEW BOT

# Multiple paths

```
cqs ./src/tests/ ./e2e/ --stack playwright
```

Audits the files a PR changed and posts one review with inline comments.

IN GITHUB ACTIONS — NO FLAGS

```
# Plain text (no ANSI — for log files)

cqs pr-review --stack playwright \
  --threshold 80

# repo, PR and token come from the env
cqs ./tests/ --output json > r.json
MANUALLY AGAINST ANY PR
cqs --read-report r.json --open

cqs pr-review --repo owner/name --pr 42 \
  --token ghp_xxx --dry-run
```

WORKFLOW PERMISSION

```
permissions:
  contents: read
  pull-requests: write
```

- `--threshold` exits non-zero below the score
- `--only-added` comment only on added lines
- `--max-comments` cap inline comments (30)

 WHAT STOPS A BAD FIX

Both agents re-audit before trusting AI output. A fix is **rejected** when it:

- introduces a new critical finding
- leaves the file unchanged
- returns no code block
- drops more than half the file

**THE CLASSIC CHEAT**

Deleting an assertion removes the finding but leaves a test that verifies nothing. Re-auditing catches it — the missing assertion is itself a critical.

**GIT SAFETY**

remediate refuses to write unless the tree is clean, so every applied fix is revertable with `git checkout`.